

Site 1 <https://www.pakwheels.com> Go to *Used Cars* → Search *Toyota Corolla* → Open any ONE listing Note down: ✓ Car Model & Year ✓ Listed Price (PKR) ✓ Mileage (km) ✓ City (where car is located) ✓ Seller Type (Dealer or Individual)

Site 2 <https://www.sbp.org.pk/ecodata/index2.asp> Find the *current USD to PKR exchange rate* ✓ Note the exact rate shown on the page ✓ Note the date of the rate

SCENARIO:

PakWheels is building a *multi-currency auction system*. You are the lead C++ developer. The system must handle car listings, bidding, currency conversion, and auction closing — all using *pure OOP principles*.

WHAT YOU MUST DESIGN:

Abstract Class: *Vehicle (Pure Abstract)* — Pure virtual method: `calculateDepreciation()` → returns `double` — Pure virtual method: `displayDetails()` → returns `void` — Attributes: `make`, `model`, `year`, `basePriceUSD`

Class: *UsedCar (Inherits Vehicle)* — Additional: `mileageKm`, `condition` (Excellent/Good/Fair), `cityLocation` — Implement `calculateDepreciation()`: Formula → $\text{depreciation} = \text{basePriceUSD} \times (\text{mileageKm} / 200000) \times 0.6$ — Implement `displayDetails()` → prints full car info

Class: *CurrencyConverter (Standalone Utility Class)* — Attributes: `usdToUKR` (the real rate you found on SBP website) — Methods: → `toPKR(double usd)` → converts USD to PKR → `toUSD(double pkr)` → converts PKR to USD — Make the exchange rate *const* — it cannot be changed after object creation (use constructor initialization list)

Class: *Bidder* — Attributes: `bidderName`, `cnic`, `budgetPKR` — Method: `placeBid(double amountPKR)` → returns `true` if bid is within budget, else `false` — Method: `displayBidder()` → prints info

Class: *AuctionSystem (Uses Composition)* — Contains: one *UsedCar* object, array of up to 5 *Bidder* objects, *CurrencyConverter* object — Method: `registerBidder(Bidder b)` → adds bidder to array — Method: `startAuction()` → asks each bidder for a bid amount, tracks highest valid bid — Method: `closeAuction()` → announces winner, prints final price in both *PKR and USD*

Demonstrate these 3 concepts explicitly:

♦ **Operator Overloading:** Overload `>` operator in *Bidder* class to compare two bidders by their budget

◆ *Friend Function*: Write a friend function `compareCars(UsedCar a, UsedCar b)` that compares two cars by their depreciated value and prints which is the better deal

◆ *Static Member*: Add a static `int totalAuctions` counter in `AuctionSystem` that increments every time a new auction object is created — print it in `closeAuction()`

REAL DATA REQUIREMENT (Mandatory):

In `main()`: — Create your `UsedCar` using the *actual PakWheels listing data* you collected — Set `usdToUKR` using the *actual SBP rate* you found — Convert the listed PKR price to USD using your converter — Write the *PakWheels URL and SBP page date* in code comments at top

Expected Output Must Show:

```
===== PAKWHEELS AUCTION SYSTEM =====
Total Auctions Running: 1
-----
Car: Toyota Corolla 2019 [Lahore]
Mileage: 45,000 km | Condition: Good
Base Price: $8,500.00 USD
Depreciation: $1,147.50
Net Value: $7,352.50 (PKR 2,047,347)
-----
Bidder 1: Ahmed Khan | Budget: PKR 2,100,000
Bidder 2: Sara Malik | Budget: PKR 1,950,000
-----
Ahmed placed bid: PKR 2,050,000 ✓
Sara placed bid: PKR 2,000,000 ✓
-----
WINNER: Ahmed Khan
Final Price: PKR 2,050,000 = $7,366.19 USD
(Rate used: 1 USD = 278.30 PKR — SBP, July 20 2026)
=====
Better Deal Analysis:
Car A depreciation: $1,147.50
Car B depreciation: $980.00
✓ Car B is the better deal!
=====
```

STRICT SUBMISSION RULES:

△ *Answer must NOT exceed 2 pages* — write tight, clean code ✓ Top of solution: paste *PakWheels URL + SBP rate + date used* ✓ Every class must have a *constructor with initialization list* ✓ Use `const` and `static` where instructed — examiner will check ✗ No

abstract class = -3 marks instantly ✗ No real website data = quiz rejected ✗ Copied work = case referred to HOD

MARKS BREAKDOWN (Total: 20)

Concept	Marks
Abstract class + pure virtual	3
Correct inheritance & override	2
Composition (AuctionSystem)	2
Operator overloading (>)	2
Friend function	2
Static member	1
CurrencyConverter with const rate	2
Real data used correctly	2
Output format + 2-page limit	2

CONCEPTS BEING TESTED: Abstract Classes • Pure Virtual Functions • Composition • Operator Overloading • Friend Functions • Static Members • Constructor Initialization Lists • const correctness • Real-world data integration